

实验二原生应用开发

项目名称：云端智能畜牧养殖管理系统

核心功能：用户登录、个人中心、数据传递

技术栈：Android (Kotlin) + iOS (SwiftUI)

开发环境：Android Studio + Xcode

1. 实验概述

1.1 实验目标

掌握 Android 原生开发 (Kotlin)

掌握 iOS 原生开发 (SwiftUI)

实现跨平台登录功能

理解 Activity/View 跳转与数据传递

掌握输入验证逻辑

完成平台差异对比分析

1.2 实验内容

环境搭建 (Android Studio、Xcode 安装)

Android 项目创建与配置

iOS 项目创建与配置

登录页面实现 (双平台)

个人中心页面实现 (双平台)

页面跳转与数据传递

功能测试

平台差异对比分析

2. 环境搭建

2.1 环境要求

Android 开发：

TRAE

Android Studio: 2022.3+

JDK: 11+

SDK: API 24+

iOS 开发:

TRAE

Xcode: 14.0+

Swift: 5.7+

macOS: 12.0+

2.2 环境搭建步骤

2.2.1 Android 开发环境搭建

访问 Android Studio 官网 (<https://developer.android.com/studio>)

下载并安装 Android Studio

安装 SDK 和模拟器

验证安装: 打开 Android Studio, 创建示例项目

2.2.2 iOS 开发环境搭建

访问 Mac App Store

下载并安装 Xcode

安装 Command Line Tools

验证安装: 打开 Xcode, 创建示例项目

2.3 环境验证

Android: 运行示例项目, 确认模拟器正常启动

iOS: 运行示例项目, 确认模拟器正常启动

3. 系统设计

3.1 系统架构

模块

职责

实现文件

Android 登录

处理用户登录验证

app/src/main/java/com/example/cif
ms/LoginActivity.kt

模块	职责	实现文件
Android 个人中心	显示用户信息	app/src/main/java/com/example/cifms/HomeActivity.kt
iOS 登录	处理用户登录验证	ios/CIFMS/ContentView.swift
iOS 个人中心	显示用户信息	ios/CIFMS/HomeView.swift

3.2 页面设计

3.2.1 登录页面

功能：用户名输入、密码输入、登录验证、错误提示

布局：居中表单布局，包含输入框、按钮和错误提示

验证规则：用户名和密码不能为空

3.2.2 个人中心页面

功能：显示欢迎信息、用户资料、退出登录

布局：居中布局，包含标题、欢迎信息和退出按钮

4. 技术实现

跨平台实现：同时实现 Android 和 iOS 两个平台的登录功能

原生开发优势：充分利用各平台的原生特性和性能优势

输入验证：实现了完善的输入验证逻辑

数据传递：掌握了不同平台的数据传递方式

用户体验：保持了各平台的设计风格 and 交互习惯

代码组织：清晰的代码结构和命名规范

5. 测试结果

5.1 测试用例

测试编号	测试场景	测试步骤	预期结果	实际结果
TC001	空输入验证	1. 不输入用户名和密码 2. 点击登录按钮	显示错误提示	符合预期
TC002	错误凭证验证	1. 输入错误的用户名或密码	显示错误提示	符合预期

测试编号	测试场景	测试步骤	预期结果	实际结果
TC003	正确凭证登录	2. 点击登录按钮	跳转到个人中心	符合预期
		1. 输入正确的用户名和密码 2. 点击登录按钮		
TC004	数据传递验证	1. 登录成功后进入个人中心 2. 查看个人中心	显示当前登录的用户名	符合预期
TC005	退出登录	1. 登录成功后进入个人中心 2. 点击“退出登录”按钮	返回到登录页面	符合预期

5.2 测试环境

Android:

模拟器: Pixel 4 API 30

Android 版本: 11.0

iOS:

模拟器: iPhone 14

iOS 版本: 16.0

5.3 测试结果

所有测试用例均通过, 系统功能正常运行。

5.4 部署与演示

5.4.1 Android 部署

构建 APK: 使用 Android Studio 构建 Debug 版本

安装到设备: 通过 ADB 或直接安装 APK

运行应用: 在设备上启动应用

5.4.2 iOS 部署

构建应用: 使用 Xcode 构建 Debug 版本

运行到设备: 通过 Xcode 部署到真机

5.4.3 演示截图

Android 登录页面



Android 个人中心



iOS 登录页面



iOS 个人中心



6. 开发规范遵循

6.1 代码规范

命名规范：

Android：遵循 Java/Kotlin 命名规范，类名使用大驼峰，变量使用小驼峰

iOS: 遵循 Swift 命名规范, 类名使用大驼峰, 变量使用小驼峰

代码风格:

Android: 使用 4 个空格缩进, 代码结构清晰

iOS: 使用 2 个空格缩进, 代码结构清晰

注释规范: 关键代码添加注释, 提高代码可读性

6.2 安全规范

输入校验: 对所有用户输入进行非空验证

敏感信息: 密码输入框使用安全输入模式

权限控制: 遵循各平台的权限管理规范

6.3 文档规范

Markdown 格式: 使用标准 Markdown 语法编写实验报告

清晰结构: 使用标题层级, 合理组织内容

代码示例: 提供完整、可运行的代码示例

7. 实验总结

7.1 完成的功能

Android 登录页面实现, Android 个人中心页面实现, iOS 登录页面实现, iOS 个人中心页面实现, 跨平台输入验证, 页面跳转与数据传递, 平台差异对比分析

7.2 技术收获

掌握了 Android Kotlin 开发基础, 熟悉了 iOS SwiftUI 开发, 理解了不同平台的页面跳转机制, 学习了跨平台数据传递方式, 掌握了输入验证逻辑的实现, 了解了原生开发的优势和挑战

7.3 系统特点

跨平台实现: 同时支持 Android 和 iOS 平台

原生体验: 充分利用各平台的原生特性

功能完整: 实现了登录、验证、跳转等核心功能

用户友好: 界面设计简洁直观

可扩展性: 预留了后续功能扩展的空间

7.4 改进方向

功能扩展：添加更多系统功能模块

数据持久化：实现用户登录状态保持

网络集成：与后端 API 对接

UI 优化：提升界面美观度和交互体验

性能优化：优化应用启动速度和运行性能

测试覆盖：增加更多测试用例

8. 附录

8.1 核心术语

术语	说明	参考链接
Kotlin	Android 官方推荐的开发语言	Kotlin 官方文档
Swift	iOS 开发语言	Swift 官方文档
SwiftUI	iOS 现代 UI 框架	SwiftUI 官方文档

8.2 开发环境

工具	版本	用途
Android Studio	2022.3.1	Android 开发 IDE
Xcode	14.3	iOS 开发 IDE
Kotlin	1.8.0	Android 开发语言
Swift	5.7	iOS 开发语言

8.3 运行说明

Android 运行： 1. 打开 Android Studio 2. 导入项目 3. 运行到模拟器或真机 4. 登录凭证： - 用户名：admin - 密码：123456

iOS 运行： 1. 打开 Xcode 2. 导入 ios/CIFMS 目录作为项目 3. 运行到模拟器或真机 4. 登录凭证： - 用户名：admin - 密码：123456

8.4 平台差异对比

对比项	Android (Kotlin)	iOS (SwiftUI)
开发语言	Kotlin	Swift
UI 框架	XML 布局 文件	SwiftUI 声明式 UI
页面跳转	Intent	NavigationLink
数据传递	Intent.put Extra	参数传递
输入验证	EditText.er ror	自定义错误提示
项目结构	分层架构	单一文件结构
构建工具	Gradle	Xcode Build System
生命周期 管理	生命周期	SwiftUI 视图生命 周期

8.5 原生开发优势

- 性能：原生应用性能更优，响应速度快
- 用户体验：符合各平台的设计规范和交互习惯
- 功能完整性：可以访问平台的所有 API 和功能
- 稳定性：系统稳定性更高，崩溃率低
- 安全性：更安全的应用沙箱和权限管理

8.6 跨平台方案对比

方案	优势	劣势
原生开 发	性能优、体验好、功能 完整	开发成本高、维护 复杂
跨平台 开发	开发效率高、代码复用	性能略差、功能受 限

选择建议：对于性能要求高、功能复杂的应用，推荐使用原生开发；对于快速迭代、多平台覆盖的应用，可以考虑跨平台方案。